

60.8. La fonction paletteFond() — les quatre nuances du fond

#include <paletteFond> : paletteFond(n0, n1, n2, n3) choisit quelle nuance de gris chaque teinte du fond prend (Game Boy d'origine).

1. Ce qu'il faut comprendre

Ce qu'elle fait : la Game Boy d'origine n'a que **4 nuances**, du plus clair (0) au plus foncé (3). `paletteFond(n0, n1, n2, n3)` dit quelle nuance prend chaque teinte des tuiles du fond. `paletteFond(3, 2, 1, 0)` **inverse** l'écran : le clair devient foncé.

La ligne à écrire : `#include <paletteFond>`.

Ses arguments : quatre nuances, de 0 à 3.

Ce qu'elle coûte : environ **70 octets** avec la mise en place de l'écran ; un appel seul, quelques octets.

Essaie : `paletteFond(0, 0, 0, 0);` : tout devient blanc, et rien n'est effacé.

On retrouve paletteFond dans 3 leçons

[61](#), [97](#), [98](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **329 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	49 o	← la console, à l'allumage	– la mise en route de la console
main()	22 o	← démarrage	– ta fonction
VBlank	8 o	← la console, à chaque image	– compte les images, 60 fois par seconde
AttendreImage	28 o	← image()	– attend l'image suivante
AttendreVBlank	16 o	← démarrage	– attend que l'écran soit libre
AttendreAcces	12 o	← EcrireTexte	– attend de pouvoir écrire une case
EcrireTexte	10 o	← texte()	– écrit un texte, lettre par lettre
CopierTuiles	67 o	← démarrage	– copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage	– vide les deux cartes de l'écran
Tuiles	96 o	← CopierTuiles	– 6 lettres sur 43
"INVERSE"	7 o	← texte()	– un texte gravé

Lettres gravées : EINRSV

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

texte()	× 1	11 o sur place, dans main()	+ EcrireTexte (10 o)
paletteFond()	× 1	4 o sur place, dans main()	
image()	× 1	3 o sur place, dans main()	+ AttendreImage (28 o)

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// ---- #include <paletteFond> : les quatre nuances du fond ----
// Le fond et les lettres échangent leurs nuances : l'écran est inversé.

#include <paletteFond> // paletteFond() : quelle nuance pour chaque teinte
#include <texte>
```

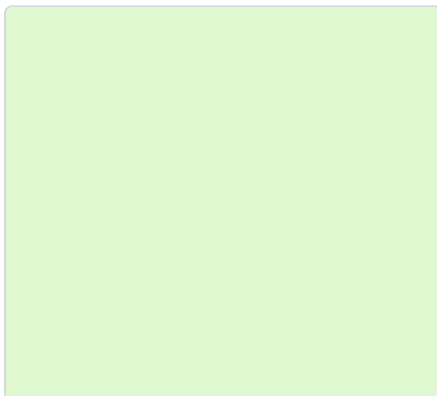
2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    texte(5, 8, "INVERSE");
    paletteFond(3, 2, 1, 0); // teinte 0 (le fond) → nuance 3, la plus foncée...

    while (true) {
        image();
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Un écran foncé, et INVERSE écrit en clair.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ le registre des nuances du fond vaut 3, 2, 1, 0 (27)

Ensuite : **61. Le panneau, et les nuances** — chapitre 9 : Ranger ce qu'on manipule. · [gameboy3](#) — écrire une cartouche Game Boy en C++